

Azure Capacity & Quota Management — Consolidated Requirements Baseline

Working document — coauthored baseline for the SaaS Design Services capacity & DR programme. This is a living requirements document. It captures the agreed architectural direction while explicitly separating confirmed baseline requirements from items that remain configurable, need proof-of-concept (POC) validation, await a business decision, or depend on an external Azure capability.

Document Control

Field	Value
Title	Azure Capacity & Quota Management — Consolidated Requirements Baseline
Version	2.2 (EU cross-geo DR region corrected; Switzerland North confirmed as the authoritative EU extension region)
Status	Working baseline — 90–95% approved direction; open POCs and business decisions remain
Baseline date	27 August 2026
Owners	Vishnuvardhan Reddy (design/requirements), Roy Szabady (strategy/business alignment)
Contributors	Azure Platform Support (Anu — deployment pipeline), Jason (quota tooling/code), Melvin Stephen (CVP — quota-as-governor strategy)
Primary consumers	Exosphere, Stratosphere, AEP, product platform teams, operations, FinOps, security & governance
Collaboration space	Atlassian Confluence (space: PI , folder <i>Azure Capacity Reservation</i>) — chosen as the shared coauthoring workspace
Source baseline	<i>Capacity reservation model for Platform brain storm</i> and <i>Weekly Connect</i> (27 Aug 2026) transcribed discussions

Version History

Version	Date	Change
2.2	27 Aug 2026	EU Geography cross-geo DR region corrected from Belgium Central to Switzerland North per REG-002 configuration review. Switzerland North confirmed as the authoritative cross-geo DR extension region for Middle East deployments. All region examples updated to reflect authoritative placement configuration.

Version	Date	Change
1.0	22-26 Aug 2026	Initial capacity reservation design flow, region categorisation, DR reserve model (~30–40%).
2.0	27 Aug 2026	Pivot to minimal bootstrap + dynamic reconciliation ; separated capacity vs quota; distributed DR; region-selection seed-record model; production-region-first onboarding; cost-driven buffer policy; Middle East DR flag.
2.1	27 Aug 2026	Added reciprocal multi-source DR hosting (DR-016), non-concurrent max-sizing (DR-017), source→destination DR index (DR-018), standby activation (DR-019), CVAL/DR co-location (PLC-010), corrected DR sizing formula (A.6), new Distributed DR Reference Model (Section 12A) and Appendix D on the sizing-formula correction.

Table of Contents

1. Purpose & Business Context
2. Strategic Drivers & Constraints
3. Design Principles
4. Terminology
5. Scope
6. Region Strategy & Classification
7. Environment Policy Requirements
8. Capacity Reservation Management Requirements
9. Quota Management Requirements
10. Combined Capacity & Quota Readiness
11. Region Selection & Customer Placement
12. Disaster Recovery Capacity Requirements
 - 12A. Distributed DR Reference Model
13. Cost Economics & FinOps Requirements
14. Provisioning & AEP Integration
15. State & Data Requirements
16. Observability & Alerting
17. Governance, Security & Compliance
18. Reliability & Non-Functional Requirements
19. Operational Requirements
20. Acceptance Criteria
21. Delivery Phases
22. Configurable Items Register
23. Pending Decisions & Mandatory POCs
24. Assumptions, Constraints & Risks

25. Final Requirement Summary
26. Appendix A — Core Formulas
27. Appendix B — Worked Cost Examples
28. Appendix C — Requirement Status Labels
29. Appendix D — DR Sizing Formula Correction (Max vs Sum)

1. Purpose & Business Context

This document defines the functional, operational, governance, security, observability, cost, and non-functional requirements for managing **Azure VM capacity reservations** and **regional VM quota** across the platform estate.

The target solution is a **Capacity & Quota Management Engine** that:

- protects production deployment capacity as the first priority;
- provides a **configurable bootstrap** position for disaster recovery instead of a fixed large reserve;
- **dynamically reconciles** reserved capacity against actual running (allocated) demand;
- **pools and allocates** regional quota across hundreds of subscriptions;
- supports **distributed DR** across multiple regions in a geography;
- supplies **fresh capacity state** to the region-selection and AEP provisioning workflows;
- minimises idle reservation cost without weakening required production and DR controls; and
- produces auditable evidence for operations and compliance (e.g., SOC 2 DR assertions).

Why this document exists. The prior write-up was a *design flow*, not a *requirements document*. Because business direction changes frequently — regions, DR scope, and customer commitments shift regularly — the team agreed to maintain a **working requirements document** as the authoritative reference that the capacity engine is built against, evolving it as decisions firm up.

2. Strategic Drivers & Constraints

These drivers shape every requirement below and explain the shift from v1 to v2.

- **Cost is now a primary constraint.** Leadership is in a cost-reduction cycle; improving margins (and, ultimately, EBITDA ahead of an IPO) is an explicit organisational goal. Large idle DR reservations (estimated in the **millions of dollars per year**) will not receive business approval, so DR must start lean and scale on demand.
- **Quota as a governor.** Per Melvin’s strategy, **quota is used as a cost and consumption governor** — it caps how much capacity a team can consume. Production maintains a quota buffer to support growth; DR quota strategy must align to the capacity-sharing model.

- **Business volatility.** Requirements evolve continually (Japan/APAC de-scoped, DHL-dedicated and Apple lost, region strategy narrowed). The design must be **flexible and configuration-driven** rather than optimised for a single fixed scenario.
- **Legal ownership of Middle East.** Legal has taken charge of the Middle East programme. Data-sovereignty constraints (a large share of customers are government/medical-associated) mean **DR is unlikely to be offered there** — cross-border DR cannot meet residency requirements.
- **Region strategy narrowed.** The active region strategy is now effectively **North America** and **Europe** focused.
- **No multi-cloud.** Multi-cloud DR has been repeatedly rejected at the ELT level; “all of Azure down” is explicitly out of scope as an addressable failure mode.
- **Preview-feature dependency.** The DR capacity model increasingly depends on **Capacity Reservation Sharing**, which is a preview feature; GA timing and supportability must be confirmed with Microsoft.

3. Design Principles

1. **Production protection first.** Prioritise production availability and the *next* approved production deployment above all else.
2. **Quota and capacity are separate resources.** Manage them independently, but validate their relationship before any deployment.
3. **Allocated demand drives reservations.** Reservation targets are computed from **allocated (running)** VMs plus a buffer — never from associated-but-deallocated VMs alone.
4. **Buffers are configurable.** Production and DR buffer values are policy inputs, not hard-coded constants.
5. **DR starts lean.** DR holds only the approved **bootstrap** capacity (enough to stand up control planes and begin recovery), not a fixed percentage copy of production.
6. **CVAL is a DR capacity source.** On a declared disaster, CVAL capacity may be shut down, disassociated, or reassigned per the approved runbook (“the first thing we do on a DR declaration is shut down CVAL”).
7. **Regional & zonal isolation.** Reservations and quota are region- and zone-bound; capacity in an unavailable region is not assumed reusable elsewhere, and reservations cannot be shared across regions or zones.
8. **Placement uses current state.** Region selection uses a recent, authoritative capacity/quota snapshot; stale state must not drive placement.
9. **Cost is a first-class constraint.** Idle reservation spend is measured, attributable, reviewed, and tunable.
10. **Automation with guardrails.** Automated changes are scoped by approved configuration, validated before execution, logged, and reversible where Azure permits.
11. **Customer placement is seeded once.** The first approved production-region decision becomes the authoritative seed for all later prod-

ucts/environments for that customer.

12. **Design for changing policy.** Geography scope, DR percentages, buffers, SKUs, regions, and environment rules are all configuration-driven.

4. Terminology

Term	Meaning in this document
Allocated VM	A running VM currently consuming compute capacity.
Associated VM	A VM linked to a Capacity Reservation Group, whether running or deallocated.
Available reserved capacity	Reserved capacity not currently consumed by allocated VMs.
Buffer target	Approved capacity held above current allocated demand for a defined scope.
Capacity Reservation Group (CRG)	Azure construct holding reservations for a defined region + availability-zone scope.
Consumer subscription	Subscription where a VM deploys and consumes a <i>shared</i> reservation owned elsewhere.
Provider subscription	Subscription that owns a reservation shared with other subscriptions.
CVAL	Customer validation environment; its capacity may contribute to DR readiness.
DR bootstrap capacity	Minimum deployed/reserved platform capacity required to initiate recovery orchestration.
Quota pool / quota group	Regional VM-family quota pooled across subscriptions and allocated on demand.
Quota hoarding	Governance practice of collecting default per-region quota into a family pool for controlled reallocation.
Seed record	Authoritative customer record holding production, CVAL, and DR regional placement.
SKU scope	The SKU/VM-family $\times$ region $\times$ zone $\times$ subscription $\times$ environment combination managed by policy.
AEP	The provisioning/automation entry point that triggers region selection and deployment.
Source region	A production region whose workload fails over on outage.
Destination (DR) region	A surviving region hosting standby DR instances for one or more source regions.
Non-concurrent sources	Multiple source regions that (per DR-001) are assumed never to fail simultaneously.

5. Scope

5.1 In Scope

- Azure VM Capacity Reservations and Capacity Reservation Groups.
- Regional and VM-family quota inventory, pooling, allocation, reclamation, and monitoring.
- Production, CVAL, and DR compute-capacity policies.
- Cross-subscription capacity reservation **sharing** where supported and approved.
- Capacity/quota state consumed by region selection and AEP provisioning.
- Automated reservation reconciliation.
- Cost, compliance, and operational evidence.
- Alerting, dashboards, audit records, and exception workflows.
- Capacity rebalancing for new deployments, growth, shutdowns, and DR exercises.

5.2 Out of Scope (this baseline)

- Application-level data replication / failover implementation (e.g., LLM/APM active-passive vs active-active — noted as a *future* consideration).
- Database and non-VM PaaS capacity mechanisms.
- Multi-cloud disaster recovery.
- Full AEP redesign beyond the required integration contracts.
- Final Middle East DR offering (pending legal/business direction).
- Permanent R&D reservations (short-lived POC reservations may be supported).
- “Entire Azure region-set / global Azure outage” as an addressable failure mode.

6. Region Strategy & Classification

The estate is organised into two capacity classes across the in-scope regions. The active strategy is **North America and Europe** focused; APAC/Australia and Japan are descoped or on hold, and Middle East is legal-owned with **no DR**.

Class	Behaviour	Notes
Standard capacity regions	Eligible for automatic selection by the placement/region-selection engine.	Default provisioning targets for production, CVAL, and DR.
Restricted deployment regions	Production-only; not auto-selected unless explicitly provided as the production region.	Used only when a customer/contract specifies the exact region.

REG-001 — Configurable region catalogue. The region classification, eligibility, and per-region flags (e.g., `DR_NOT_OFFERED`, zone support, restricted) must be configuration-driven and versioned.

REG-002 — Example correction discipline. Region examples must be sourced from authoritative configuration, not slideware (e.g., “Belgium” was corrected to **Switzerland North** during review). Placement config is the single source of truth.

REG-003 — Multi-region distribution benefit. Three-to-four regions per geography is a design goal because it distributes customer workloads and materially reduces the DR capacity that must be reserved per region. A **two-region** model is explicitly identified as problematic — it cannot guarantee sufficient failover capacity if all production concentrates in one location.

7. Environment Policy Requirements

ENV-001 — Production reservation coverage. Manage approved production VM SKUs via CRGs wherever production protection is enabled.

ENV-002 — Production-only initial enforcement. Mandatory reservations apply to **production** first. Non-production reservation enforcement stays configurable because non-prod VMs may be deallocated for cost savings (e.g., site teams powering down non-prod). The design must avoid a state where non-prod reservations block cost-driven deallocation.

ENV-003 — Hard separation constraints. The following placement constraints are baseline: - Non-prod and prod **cannot** share capacity. - DR and prod **cannot** share capacity. - DR **may** share with non-prod.

ENV-004 — CVAL treatment. Treat CVAL as a potential DR capacity source. The system identifies CVAL reservations, allocated/associated CVAL VMs, releasable capacity, the production portion depending on that CVAL location, and the actions required to free capacity for DR.

ENV-005 — DR bootstrap, not full duplicate. DR must **not** default to a fixed 30–40% copy of production. Support a configurable bootstrap target by workload, product, region, zone, VM family, and subscription model.

ENV-006 — DR bootstrap cannot be implicitly zero. A zero DR target is allowed only via explicit approved policy. Separately identify any already-running DR control-plane/skeleton capacity that satisfies the bootstrap requirement (so we never rely on capacity that isn’t actually there).

ENV-007 — R&D policy. No permanent R&D reservations by default; support time-bound reservations for POCs, latency testing across US regions, DR exercises, and engineering tests.

8. Capacity Reservation Management Requirements

CAP-001 — Authoritative managed scope. The engine operates **only** on resources declared in an approved configuration source (the “scope file”). Scope includes: tenant/management scope, subscription, region, availability zone, resource group, CRG, reservation name, SKU/VM-family, environment, buffer policy, enabled/disabled state, and effective date/version. Anything outside scope is never modified automatically.

CAP-002 — Azure resource must precede config activation. A managed reservation is not activated in deployment config until the corresponding Azure reservation and CRG exist and validate. **Any change starts in Azure first, then the config** — never the reverse. This prevents deployment failures from referencing a non-existent reservation.

CAP-003 — Reservation target formula.

Target Reserved Capacity = Allocated VM Count + Configured Buffer

The target is **never** computed from associated VM count alone.

CAP-004 — Associated-but-deallocated VMs. These do not automatically force reservation retention. Report them separately so teams understand restart risk and cost. (Rationale: if a customer such as a non-paying account is shut down, its VMs become *associated* not *allocated*, and the engine must not keep paying to guarantee that capacity.)

CAP-005 — Automated reconciliation. Periodically compare reserved quantity, allocated VMs, associated VMs, available reserved capacity, and buffer target; then raise or lower the reservation toward the approved target, subject to Azure availability, policy, and change controls.

CAP-006 — Reconciliation frequency (configurable). The reference implementation is a container-app job packaged as a Docker image that runs **every 6 minutes**; the production interval must be tunable against API throttling, operational risk, cost, and deployment responsiveness.

CAP-007 — Scale-up behaviour. When allocated demand rises, attempt to raise reserved capacity to restore the buffer. If Azure cannot supply it, hold the current safe state, **raise an alert**, and expose the buffer deficit (so it can be negotiated with Microsoft).

CAP-008 — Scale-down behaviour. When allocated demand falls, reduce excess reservation toward the buffer, after applying any minimum-hold interval, DR protection, approved maintenance exclusion, and cost policy.

CAP-009 — Zero-capacity support. Where Azure permits, reduce an unused managed reservation to **zero** rather than deleting the reservation object (“set it to zero, don’t delete it”).

CAP-010 — No automatic deletion by default. Normal reconciliation

never deletes CRGs or reservation definitions; deletion uses a separate approved decommissioning workflow.

CAP-011 — Availability-zone isolation. Track and manage reservations by region **and** availability zone; zone-1 capacity is not counted as available in another zone.

CAP-012 — Regional isolation. Reservations are never counted, moved, or shared across regions. DR planning models destination-region capacity independently.

CAP-013 — Capacity sharing. Support cross-subscription reservation sharing within the supported Azure scope (same region **and** same availability zone; up to ~100 subscriptions per tenant) once approved. Track provider/consumer subscriptions, sharing permissions, consumption, and revocation.

CAP-014 — Shared capacity visibility. For each shared reservation, show owning subscription, authorised consumers, consuming VMs, consumed quantity, remaining quantity, region/zone, SKU, and sharing state.

CAP-015 — No double counting. Capacity shared to multiple subscriptions is counted once at the provider and apportioned by actual consumer allocation; *authorisation to consume* is not treated as *allocated* capacity.

CAP-016 — Reservation integrity validation (pre-deploy). Before deploying against a reservation, validate: reservation exists; SKU matches; region matches; zone matches; consumer subscription authorised (if shared); sufficient reserved capacity or approved over-allocation; and required quota available in the deploying subscription.

CAP-017 — Deployment failure policy. If reservation enforcement is mandatory and validation fails, the deployment **fails safely** with a clear reason (Anu's pipeline fails rather than silently deploying without the reservation). No silent deployment without the required reservation unless an approved break-glass policy is invoked.

CAP-018 — Over-allocation policy. Support an explicit policy to associate more VMs than reserved where operationally required, distinguishing **guaranteed allocated** capacity from **associated-but-unguaranteed** capacity. The engine tracks the *allocated* number so it can over-associate while still guaranteeing what is actually running.

CAP-019 — Scope-file governance. Adding a SKU to management requires (a) creating the Azure reservation and (b) adding it to the scope file; removing a SKU from the file removes it from engine management. Both sides must stay consistent (the deployment pipeline reads the same file to decide reservation association).

9. Quota Management Requirements

QUA-001 — Separate quota domain. Manage quota as a separate control plane from reservations. A reservation is **not** proof that deployment quota exists — you can hold a reservation and still fail to deploy without quota.

QUA-002 — Regional & family scope. Maintain quota inventory by subscription, region, VM/quota family, assigned quota, usage, available quota, pooled quota, and pending increases.

QUA-003 — Central pooling (“quota hoarding”). Where Azure quota groups support the scope, eligible subscriptions contribute **unused** regional VM-family quota to a centrally governed pool. Azure allocates a default (e.g., ~350 vCPU) per region to every subscription; unused cross-region quota (e.g., Switzerland quota sitting in a US subscription) is collected into the family pool for controlled reallocation. This is what enables fast turnaround of quota requests (“I’m not requesting quota, I already have it — I’m just allocating it”).

QUA-004 — Single governed pool per supported scope. Prefer **one** governed quota group per applicable regional/quota-family scope (prod, non-prod, and DR together) to maximise manipulation flexibility, unless Azure limits or governance boundaries require separation. Final grouping stays configuration-driven.

QUA-005 — Quota as cost/consumption governor. Use quota allocation to cap deployable capacity by product, environment, subscription, region, and VM family. Do **not** increase a subscription’s quota merely because unallocated pooled quota exists — teams must justify need (a request for thousands of cores is not trivial and requires a stated reason).

QUA-006 — Production growth buffer. Production subscriptions maintain a configurable quota headroom above current usage to support growth and the next deployment (e.g., a team using 10k of 15k cores that grows to 14k triggers a quota top-up to preserve the buffer).

QUA-007 — Quota-reservation validation. For each managed scope, validate that quota is sufficient to deploy against the intended reservation. The invalid condition to prevent:

Required Deployment Quota > Available Quota in the Deploying Subscription

Quota may exceed reserved capacity; reserved capacity exceeding deployable quota is surfaced as a readiness **risk** (reservation without quota cannot deploy).

QUA-008 — Dynamic allocation. Allocate quota from the pool to a target subscription per policy, deployment demand, production buffer, and DR need.

QUA-009 — Quota reclamation. Reclaim unused subscription quota into the pool when policy permits, never dropping a subscription below current usage, committed demand, production buffer, or approved DR need.

QUA-010 — Quota request governance. Increase requests record justification, target workload, SKU/family, region, amount, existing usage, target date, and owner. Requests without sufficient justification are not auto-escalated to Microsoft. (Future: automate intergroup quota requests, building on Jason’s existing code.)

QUA-011 — Quota source discovery. Discover reusable quota assigned by default to subscriptions in regions where they will not deploy, subject to safe-reclamation checks.

QUA-012 — Region-failure assumption. Quota bound to a failed region is **not** assumed transferable to the DR region. DR destination quota must be planned and present in the destination region. Quota, quota groups, and reservations are all region-scoped.

QUA-013 — Consumer-subscription quota (POC-gated). Assume a VM deployment requires quota in the **consumer/deploying** subscription even when capacity is *shared* from a provider subscription. Reading to date indicates **quota lives at the subscription level, not the reservation group** — a consumer needs its own quota even though the shared reservation comes from the provider. This assumption stays **POC-validation-required** until confirmed by test and authoritative Azure guidance for the selected feature version.

QUA-014 — Quota allocation audit. Log every allocation, reclamation, request, approval, rejection, and failed change with before/after values, actor/workload identity, policy reason, correlation ID, and timestamp.

10. Combined Capacity & Quota Readiness

RDY-001 — Deployment readiness gate. A deployment is capacity-ready only when all pass: target region approved; zone supported; SKU supported; reservation policy known; reservation exists (if required); sufficient reservation or approved over-allocation; sufficient consumer-subscription quota; records fresh enough; and no blocking policy/exception.

RDY-002 — Readiness states. Expose a machine-readable state: `READY`, `READY_WITH_RISK`, `QUOTA_DEFICIT`, `RESERVATION_DEFICIT`, `CAPACITY_UNAVAILABLE`, `STALE_STATE`, `POLICY_BLOCKED`, `VALIDATION_REQUIRED`.

RDY-003 — Capacity and quota must understand each other. Though controlled separately, correlate them by region, zone, SKU/family, subscription, environment, and intended demand — capacity and quota must be **balanced** (quota = reservation for any SKU we intend to scale).

RDY-004 — No stale placement. Region selection/provisioning must not use a snapshot older than the configured max age; if stale, refresh synchronously or stop with `STALE_STATE`. (The static store may be updated daily/weekly and feeds the next selection cycle; deployments between updates must not act on an outdated region-capacity state.)

11. Region Selection & Customer Placement

PLC-001 — Production region is the primary input. The default onboarding path requires the customer or platform workflow to supply the exact Azure production region. ACRME validates the supplied region against Hard Constraints and placement-policy rules instead of deriving a region from a broad geography. Rationale: requiring a precise region avoids “I picked North America but meant East Coast” churn and the contract rewrites it causes.

PLC-002 — Geography-based selection is exceptional. Geography-only selection (North America / Europe / Middle East / APAC → engine derives the region) is an exceptional path. It requires explicit exception approval and customer acknowledgement that the derived production region becomes fixed until an approved migration changes the seed. Without an approved exception, the exact-region default (PLC-001) applies.

PLC-003 — Customer seed record. The first placement decision creates an authoritative seed record: customer/realm identifier, geography, production region, CVAL region, DR region (or NOT_OFFERED), products covered, decision timestamp, policy/engine version, capacity-snapshot reference, exception reference (if any), and approval metadata.

PLC-004 — Reuse across products. Subsequent products/environments for the same customer + geography read the seed record instead of re-selecting production (“their production is their production for all products in that geography”). The engine is **not** re-invoked per product once seeded.

PLC-005 — Controlled seed change. The seed is never regenerated on upgrades, rebuilds, or routine deployments; changes require an approved migration/exception workflow with impact analysis.

PLC-006 — CVAL & DR selection. Once production is fixed, the engine selects/validates CVAL and DR using current readiness, separation policies (ENV-003), regional restrictions, workload distribution, and capacity weighting.

PLC-007 — Live weighting. Placement weighting considers allocated capacity, available reservation, quota availability, buffers, prod/CVAL distribution, expected DR contribution, zone support, regional restrictions, and state freshness.

PLC-008 — Lowest suitable load. Select the lowest-risk suitable location per the weighted policy — the lowest-consumption subscription for the new customer’s CVAL/DR — not merely the lowest raw utilisation. If the chosen production region lacks capacity, raise an alarm/exception while still placing CVAL/DR appropriately.

PLC-009 — AEP-triggered pipeline. Region selection is the **first** pipeline AEP triggers; it emits production → derives DR (and CVAL) and writes the seed. Implementation is a function-app flow (not a logic app) invoked via API or GitHub Actions.

PLC-010 — CVAL/DR co-location. A customer’s CVAL and DR **may co-locate in the same destination region** to support the CVAL-sacrifice bootstrap pattern (DR-005/DR-006). When co-located, the engine must record that the CVAL capacity is earmarked as releasable toward that customer’s DR activation, and must not double-count it as both live CVAL and available DR headroom.

12. Disaster Recovery Capacity Requirements

DR-001 — Single-region failure basis. The default model plans for failure of **one** production region within a geography. Simultaneous multi-region failure is outside the default guaranteed model unless separately funded/approved (“if we have DR in two regions in a geography, we’re in serious trouble”).

DR-002 — Distributed DR. DR capacity is computed from the **portion** of the source region’s production workload assigned to each destination — not by reserving the full source workload in every destination. Because a customer’s workload is distributed across the 3–4 regions in a geography, only that customer’s *portion* needs protecting per destination.

DR-003 — Destination distribution. Record how each source region’s recoverable workload distributes across eligible destination subscriptions, regions, zones, and SKUs.

DR-004 — CVAL target contribution. Normal CVAL placement should contribute toward the capacity needed for the production portion expected to fail over to that location (“CVAL only needs to support a *portion* of a production failover, not all of it”).

DR-005 — CVAL may exceed DR need. CVAL capacity may exceed the minimum DR contribution (it’s “free” while running); excess running CVAL is potentially reusable during a declared disaster, subject to shut-down/disassociation rules.

DR-006 — Staged DR capacity acquisition. The recovery process supports staged expansion: 1. Use approved **bootstrap / pre-staged headroom**. 2. Allocate available reservation + quota already in the destination. 3. **Shut down / disassociate** eligible CVAL workloads to free capacity. 4. Share or reassign reservations within supported region/zone boundaries. 5. Allocate pooled quota to DR subscriptions. 6. Request additional Azure quota/capacity where required. 7. Report unrecoverable capacity gaps.

This staging lets priority (P0/P-1) customers start immediately from bootstrap headroom while rebalancing proceeds, rather than waiting on deallocation/disassociation APIs that will be **throttled** industry-wide during a regional event.

DR-007 — DR target is configurable. Bootstrap quantity may be a node count, SKU-specific quantity, vCPU value, workload tier, or approved percent-

age; configurable by product and region. (Design started at ~30–40%, then discussed 10–20%, then ~5%, converging on “whatever is needed to bootstrap and is used = effectively free.”)

DR-008 — Control plane first. Bootstrap prioritises required platform control planes and recovery orchestration before customer workload waves (“without a control plane, nothing else runs”).

DR-009 — Recovery prioritisation. Support prioritised customer/workload recovery waves using **authoritative** business priorities; the engine does not invent priority.

DR-010 — DR declaration guardrail. Destructive/service-impacting actions against CVAL occur only after an authorised DR declaration or approved DR exercise trigger.

DR-011 — Source region not a capacity source during outage. During a destination recovery decision, do not count reservations/quota from the unavailable source region. Cross-region reuse does not work, and a down region’s VMs/quota remain bound as left (validated behaviour, incl. the Australia outage where VMs were not truly deallocated behind the scenes).

DR-012 — DR drill rotation. Support periodic DR drills and role swaps (active recovery) without losing the seed, historical capacity state, or audit trail. CVAL capacity “flips around” per region during the annual drill.

DR-013 — Failback policy (configurable). Support both an **extended DR run** (~1 year, preferred) and an **earlier failback** (~30 days) model to prove failback. Duration/execution is a business/operations decision.

DR-014 — Middle East policy flag. Support `DR_NOT_OFFERED` per country/region where legal/data-sovereignty prevents an acceptable DR design (current legal position for Middle East). Middle East production may still exist; DR does not.

DR-015 — Future active-active consideration. Note (out of baseline scope) that LLM/APM may move from active-passive to active-active multi-region in future; the capacity model should not preclude it.

DR-016 — Reciprocal multi-source hosting. Every region may concurrently perform **three roles**: (a) Production for its own customers, (b) CVAL host for customers whose production is elsewhere, and (c) standby **DR host for customers originating in multiple *different* source regions**. The design must support this many-to-many topology — e.g., Region 1’s DR block simultaneously holds Cust2 (prod in R2) and Cust7 (prod in R3). DR distribution is therefore **bidirectional**, not a single source→destination fan-out.

DR-017 — Non-concurrent capacity sharing (max, not sum). Because only one region fails at a time (DR-001), a region that serves as DR target for several source regions must be sized to absorb the **largest single source** it

protects, **not the sum of all of them**. This permits **shared / overcommitted DR capacity** across mutually-exclusive failure events and is the primary mechanism that keeps the lean DR model affordable (see Appendix A.6 and Appendix D). Reserving the sum would over-provision and negate the cost savings that justified the bootstrap model (FIN-006).

DR-018 — Source→destination DR index. Maintain an authoritative **bidirectional mapping** that records, for each source region, which destination regions hold its customers’ DR instances and in what quantity/SKU. On a regional failure the engine uses this index to determine exactly which standby instances to activate and where. The index is the reverse view of the per-customer seed record (PLC-003) and is a required entity in the state store (DAT-002).

DR-019 — Standby activation on declaration. On an authorised DR declaration (DR-010), the engine shall transition the failed region’s customers’ pre-placed DR instances from **associated** → **allocated** (inactive/standby → active) in approved business-priority order (DR-009), acquiring capacity via the staged sequence in DR-006 (bootstrap headroom → available reservation/quota → CVAL sacrifice → sharing/reassignment → pooled quota → Azure request). Activation state per customer must be tracked, auditable, and reversible on failback (DR-013).

12A. Distributed DR Reference Model

This section formalises the failover topology reviewed against the distribution diagram. It is the canonical picture the engine (DR-002/003/016/017/018/019) implements.

12A.1 Topology

- A **geography** contains **N regions** (target: 3–4; minimum viable: 3 for safe distribution — see REG-003).
- Each region simultaneously hosts three stacked blocks: **Prod**, **CVAL**, and **DR** (DR-016).
- A customer’s **Prod**, **CVAL**, and **DR** live in **different** regions per the separation rules (ENV-003), though **CVAL** and **DR** of a given customer **may co-locate** (PLC-010).
- Each region’s **DR block is a standby (associated, not allocated)** landing zone for customers whose **Prod is in other regions** (green = active/allocated; red = inactive/associated in the diagram).

12A.2 Worked example (from the reviewed diagram)

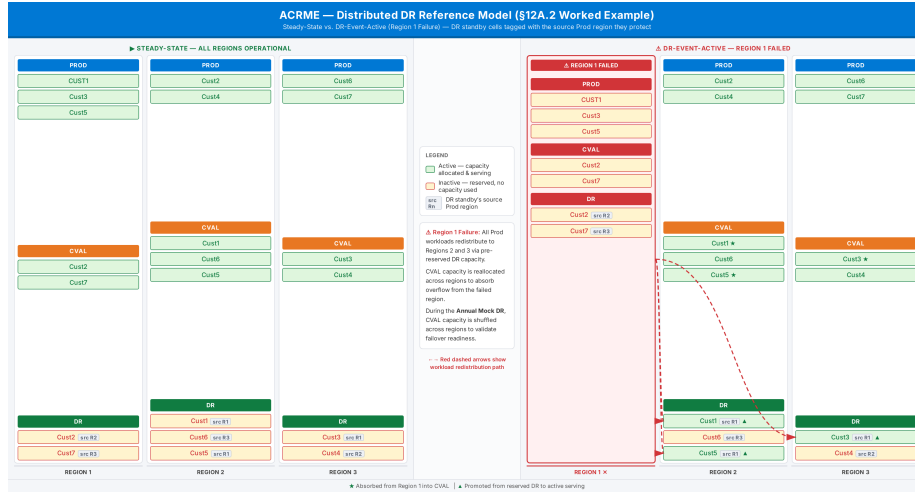


Figure 1: ACRME Distributed DR Reference Model

Region	Prod customers	CVAL hosted	DR standby hosted (source region)
Region 1	Cust1, Cust3, Cust5	Cust2, Cust7	Cust2 (R2), Cust7 (R3)
Region 2	Cust2, Cust4	Cust1, Cust6, Cust5	Cust1 (R1), Cust6 (R3), Cust5 (R1)
Region 3	Cust6, Cust7	Cust3, Cust4	Cust3 (R1), Cust4 (R2)

12A.3 Failover behaviour (Region 1 goes down)

1. Region 1's **production** customers (Cust1, Cust3, Cust5) are activated on their **pre-placed DR** instances in the surviving regions (DR-018 index lookup): Cust1 → R2, Cust5 → R2, Cust3 → R3.
2. Standby DR instances flip **associated** → **allocated** in priority order (DR-019), acquiring capacity via the staged sequence (DR-006), including **sacrificing CVAL** in the destination if required (DR-005/DR-010).
3. **Capacity and quota shift** across regions accordingly; the state store is updated and feeds the next selection cycle (DAT-001, RDY-004).
4. Region 1's own **DR-standby** duties for other regions (it was holding Cust2/Cust7 standby) are understood to be **temporarily unavailable** while Region 1 is down — acceptable under the single-failure assumption (DR-001), because R2/R3 are not simultaneously failing.

12A.4 Annual mock DR

During the yearly drill (DR-012), **CVAL capacity per region is reshuffled** to validate the failover paths and to keep destination reservations right-sized for the production portion each region protects.

13. Cost Economics & FinOps Requirements

FIN-001 — Idle cost measurement. Calculate/ingest the cost of unused reserved capacity by region, SKU, subscription, environment, and owner.

FIN-002 — Configurable economic policy. Buffer and bootstrap values are adjustable **without code changes** to balance risk, deployment speed, audit needs, and cost.

FIN-003 — Cost before expansion. Before increasing a persistent buffer or DR target, expose expected incremental cost and require justification.

FIN-004 — Cost allocation. Reservation cost is attributable to the owning platform/product/environment or centrally funded DR function per approved tagging/chargeback policy.

FIN-005 — Underutilisation review. Long-running underutilised reservations create a review item rather than being silently retained. (The cost-optimisation team already chases orphaned reservations left behind after maintenance — set to zero, don't delete.)

FIN-006 — No cost-only unsafe reduction. Cost optimisation never reduces reservations/quota below allocated demand, committed production buffer, or authorised DR bootstrap without an approved exception.

FIN-007 — Audit-friendly flexibility. Buffer/bootstrap numbers must be adjustable to satisfy SOC 2 DR assertions (“here’s how we ensure DR capacity”) while still optimising cost — the number is a tunable governance lever, not a fixed architectural constant.

FIN-008 — Shared-DR overcommit accounting. Where DR-017 sizing (max-not-sum) relies on non-concurrent sharing, the cost model must reflect the **shared/overcommitted** reservation as a single cost, not per-source duplication, and must flag the residual risk if the single-failure assumption is ever violated.

Why lean DR won. Cost modelling during review showed a 30% empty DR reserve is prohibitively expensive at platform scale — on the order of **millions per year** — and would likely cause leadership to cancel multi-region entirely. See Appendix B for worked examples and Appendix D for the max-vs-sum saving.

14. Provisioning & AEP Integration

INT-001 — Capacity API/workflow. AEP calls a capacity+placement service/pipeline before the first relevant environment deployment and before any later deployment that changes capacity demand.

INT-002 — Idempotent interface. Capacity checks and reservation operations are idempotent; repeated calls with the same correlation ID + desired state do not duplicate allocations or records.

INT-003 — Required input. customer/realm; product; environment; production region (or approved exception input); subscription; SKU + count; zone requirement; deployment priority; requested time; correlation ID.

INT-004 — Required output. resolved prod/CVAL/DR placement; readiness status; approved reservation/CRG reference; quota status; available vs required quantities; blocking reasons; exception requirements; snapshot timestamp; trace/decision ID.

INT-005 — Deployment reservation association. When policy requires it, associate the VM / VM scale-set instance configuration with the validated reservation reference (Anu's pipeline supplies the reservation-group name when SKU + subscription match).

INT-006 — Concurrent deployment control. Prevent two concurrent decisions from consuming the same last unit of capacity/quota via reservation-of-intent, optimistic concurrency, or equivalent.

INT-007 — Partial-failure handling. If quota is allocated but deployment/association fails, record the partial state and either compensate safely or raise a recoverable operational task.

15. State & Data Requirements

DAT-001 — Authoritative state store. Maintain an authoritative operational store (e.g., a storage-account table) separate from transient API responses; it becomes the input to the next region-selection cycle.

DAT-002 — Minimum entities. subscriptions; regions/zones; SKUs/quota families; CRGs/reservations; provider/consumer sharing relationships; allocated/associated VMs; quota pools + subscription assignments; buffers/policies; customer seed records; **source→destination DR index (DR-018)**; DR distribution plans; deployment intents; reconciliation runs; alerts/exceptions; audit events.

DAT-003 — Freshness metadata. Every observation carries collection time, source, region, subscription, and status; derived readiness references the source observations used.

DAT-004 — Historic state. Retain history sufficient to explain capacity growth, quota changes, reservation cost, failed placements, DR readiness, and

policy changes.

DAT-005 — Configuration versioning. Scope files/policies are version-controlled; each reconciliation/deployment decision records the config version used. (Reference repo: `plat-saasa-azure-capacity-dev`.)

DAT-006 — Schema compatibility. API/config schema changes are versioned and backward compatible for an approved transition period.

16. Observability & Alerting

OBS-001 — Core metrics. reserved quantity; allocated VM count; associated VM count; available reserved; configured buffer; buffer deficit/surplus; quota assigned/used/available; pooled quota available; reservation utilisation %; estimated unused reservation cost; reconciliation success/failure; Azure API throttling/latency; state age; deployment blocks; DR capacity coverage; **per-destination DR max-source coverage (DR-017)**.

OBS-002 — Required alerts. production buffer below target; DR bootstrap below target; quota below required/growth buffer; reservation quantity > deployable quota; inability to raise a reservation (needs Microsoft); Azure throttling causing stale state; config references missing Azure resources; unauthorised/unexpected sharing; reconciliation failures; stale placement data; prolonged unused reservation cost; desired-vs-actual divergence; **destination DR coverage below its max protected source**.

OBS-003 — Alert context. region, zone, subscription, SKU/family, environment, desired value, actual value, detection time, correlation ID, and recommended owning team.

OBS-004 — Dashboards. regional, product, subscription, SKU, environment, and DR views; production readiness, DR readiness, quota-pool health, and idle cost separately visible; **source destination DR mapping view**.

OBS-005 — SLO reporting. availability, reconciliation latency, data freshness, decision latency, and failed-automation rates against approved SLOs.

17. Governance, Security & Compliance

GOV-001 — Least privilege. Workload identities receive only the roles/permissions required for inventory, quota management, reservation management, sharing, and deployment validation.

GOV-002 — Separation of duties. Policy approval, production execution, exception approval, and audit review are separable roles.

GOV-003 — Scoped automation. Automation is constrained by tenant, management group, subscription, resource group, region, zone, SKU, and the scope file.

GOV-004 — Change approval. High-impact changes — reducing production protection, setting DR bootstrap to zero, deleting reservations, broad sharing, reclaiming committed quota — require approval.

GOV-005 — Break-glass controls. Break-glass requires authorised identity, reason, bounded scope, expiry, and full audit logging.

GOV-006 — Audit evidence. Produce evidence of how production/DR capacity is maintained, how deficiencies are detected, and how changes are controlled (SOC 2-ready).

GOV-007 — Policy exceptions. Each exception has an owner, reason, approved scope, start/expiry dates, compensating controls, and review status.

GOV-008 — Secrets & credentials. No secrets in config files or logs; prefer managed/workload identity.

GOV-009 — Data classification. Customer placement/workload metadata is classified and protected per enterprise standards; telemetry avoids unnecessary customer-sensitive content.

18. Reliability & Non-Functional Requirements

NFR-001 — Availability. The capacity decision path is not an uncontrolled single point of failure for production provisioning; degraded-mode behaviour is defined. **NFR-002 — Consistency.** Concurrency control prevents double-committing capacity/quota. **NFR-003 — Performance.** Readiness responses meet an approved latency target; long Azure changes return a tracked operation state rather than blocking. **NFR-004 — Scale.** Operates across **hundreds** of subscriptions, multiple regions/zones, VM families, products, and seed records. **NFR-005 — API-throttling resilience.** Use batching, caching, backoff, jitter, retry limits, and per-scope rate control; avoid API storms during a regional event. **NFR-006 — Recoverability.** State store, config, and audit trail support backup/restore and reconstruction of desired state. **NFR-007 — Idempotency.** All mutating operations are idempotent or protected by a durable operation key. **NFR-008 — Testability.** Policies, formulas, reconciliation, weighting, failover distribution, and compensation are independently testable. **NFR-009 — Simulation mode.** Support read-only/dry-run showing intended reservation/quota/placement/cost changes without applying them — including **DR failover simulation** per 12A. **NFR-010 — Explainability.** Every decision exposes policy inputs, state snapshot, formula, and reason code.

19. Operational Requirements

OPS-001 — Runbooks for: production buffer deficit; quota exhaustion; reservation scale-up failure; stale state; deployment blocked by missing reservation; DR declaration; CVAL shutdown/disassociation; quota allocation to DR; reservation sharing activation/revocation; reconciliation rollback/pause; man-

ual emergency override; regional recovery and failback; **standby DR activation sequence (DR-019)**. **OPS-002 — Safe pause.** Operators can pause mutation while retaining inventory and alerting. **OPS-003 — Manual override.** Authorised operators set temporary desired values with expiry + reason; the engine never overwrites an active approved override. **OPS-004 — Maintenance-window awareness.** Planned maintenance changing VM allocation/association is visible to the engine to avoid inappropriate scaling or alert noise. **OPS-005 — Ownership.** Every region, subscription, reservation scope, quota pool, alert, and exception has an owning team and escalation route.

20. Acceptance Criteria

The baseline is implementable when all are demonstrated:

1. Inventory of reservations, allocated VMs, associated VMs, quota, sharing, and configuration across an approved scope.
2. Engine computes **allocated + buffer** and reconciles reservation quantity.
3. A deallocated-but-associated VM does not unnecessarily preserve reservation quantity unless policy requires.
4. Detects a missing Azure reservation referenced in config **before** production deployment.
5. Blocks or safely handles a deployment with insufficient consumer-subscription quota.
6. Allocates and reclaims pooled quota without dropping any subscription below protected requirements.
7. Prevents concurrent requests from double-committing capacity or quota.
8. Exposes a fresh, machine-readable readiness result to AEP/provisioning.
9. Creates and reuses a customer placement seed record.
10. Simulates a single-region DR event and computes **distributed destination requirements using max-not-sum sizing (DR-017)**.
11. Identifies releasable CVAL capacity for DR and requires an authorised trigger before service-impacting action.
12. Reports idle reservation cost and protection deficits.
13. Records complete audit events for automated and manual changes.
14. Enters a safe degraded state during Azure API throttling and reports stale data.
15. Supports dry-run evaluation before applying policy changes.
16. **Maintains and queries the source→destination DR index (DR-018) and activates the correct standby set (DR-019) for a simulated single-region failure.**

21. Delivery Phases

Phase 1 — Capacity Visibility & Production Protection

Read-only capacity/quota inventory · managed scope file · production reservation reconciliation (**allocated + buffer**) · missing-resource validation · alerts, audit logging, dashboards, dry-run · AEP readiness API. **Objective:** protect production first.

Phase 2 — Quota Pool Automation

Quota-group inventory · dynamic allocation & reclamation · production growth buffer · quota-request workflow · correlated quota-reservation readiness. **Objective:** treat quota as a centrally governed resource (quota-as-governor).

Phase 3 — Placement & Customer Seed

Exact production-region input · seed-record creation & reuse · CVAL/DR weighted placement · **CVAL/DR co-location (PLC-010)** · concurrent-deployment controls · capacity commitment workflow. **Objective:** deterministic customer placement.

Phase 4 — Distributed DR Capacity Management

Destination workload distribution · **source→destination DR index (DR-018)** · **reciprocal multi-source hosting (DR-016)** · **max-not-sum sizing (DR-017)** · CVAL release modelling · bootstrap & recovery waves · **standby activation (DR-019)** · DR declaration workflow · capacity sharing + DR quota allocation (post-POC) · DR simulation & compliance evidence. **Objective:** enterprise-scale DR capacity governance.

22. Configurable Items Register

#	Item	Current direction	Status
C-1	DR bootstrap capacity	Lean, “enough to bootstrap”; discussed 30–40% → 10–20% → ~5% → used-is-free	Configurable — no fixed value
C-2	Production reservation buffer	allocated + buffer ; ref impl buffer = 1 in dev	Configurable by prod-uct/region/env/SKU
C-3	Production quota buffer	Headroom above usage to support growth	Configurable — value TBD
C-4	Reconciliation frequency	Ref impl every 6 minutes	Configurable — prod value TBD

#	Item	Current direction	Status
C-5	Failback model	Prefer ~1 year run; ~30-day failback alternative	Business decision
C-6	Onboarding selection mode	Exact production region (default); geography (exception)	Configurable + exception policy
C-7	Quota grouping model	One governed pool preferred	Configurable
C-8	Region catalogue & flags	NA + Europe focus; restricted/standard classes; DR_NOT_OFFERED	Configurable
C-9	Reservation over-allocation	Track allocated; allow over-association	Configurable policy
C-10	DR drill duration/rotation	Annual drill; role flip	Business decision
C-11	DR sizing basis	Max over non-concurrent sources (DR-017); sum available as conservative override	Configurable — max is default

23. Pending Decisions & Mandatory POCs

ID	Item	Required outcome
POC-001	Capacity Reservation Sharing quota behaviour	Confirm whether the consumer subscription must hold the quota when consuming a shared reservation; document observed Azure behaviour. (Top technical unknown.)
POC-002	Sharing consumption order	How Azure allocates shared capacity when multiple consumers request the same SKU.
POC-003	Zone/subscriptions boundaries	Validate supported sharing combinations for the selected feature version.
POC-004	Change under throttling	Safe reconciliation + retry behaviour during Azure API throttling.
POC-005	VM association/shutdown states	Reservation/guarantee behaviour across allocated/stopped/deallocated/associated/disassociated.

ID	Item	Required outcome
POC-006	DR subscription topology	Dedicated DR subscription+cluster vs shared production subscription model.
POC-007	Bootstrap sizing	Minimum bootstrap per product incl. required control-plane nodes/SKUs.
POC-008	Production quota buffer	Initial buffer policies by product/VM family.
POC-009	Production reservation buffer	Initial reserved-capacity buffer by product/region/zone/SKU.
POC-010	Reconciliation interval	Production interval after API + cost testing.
POC-011	Max-not-sum overcommit safety	Validate that shared/overcommitted DR reservations (DR-017) behave correctly when a standby set activates, and quantify residual risk if two regions ever fail together.
DEC-001	Middle East DR offering	Record legal/business decision per country/geography (DR_NOT_OFFERED).
DEC-002	DR drill duration & failback	Extended run vs earlier failback.
DEC-003	Geography exception approval	Approver + binding customer acknowledgement for geography-only onboarding.
DEP-001	Azure feature maturity	Track Capacity Reservation Sharing preview→GA status and approved enterprise-usage conditions.

24. Assumptions, Constraints & Risks

Assumptions

- Quota lives at the subscription level, not the reservation group (pending POC-001).
- A down region's VMs/quota/reservations remain bound as left and are not reusable during the outage.
- Customer workloads distribute across 3–4 regions per geography, enabling portion-based DR.
- **Only one region in a geography fails at a time** — the basis for max-not-sum DR sizing (DR-001/DR-017).

Constraints

- Reservations cannot be shared across regions or availability zones.
- Any reservation change must start in Azure before the scope file.

- Non-prod/prod and DR/prod cannot share capacity; DR may share with non-prod.
- No multi-cloud DR; global Azure outage is not an addressable failure mode.

Key Risks

Risk	Impact	Mitigation
Sharing quota behaviour unknown	Blocks DR cost model	POC-001 + Microsoft confirmation
Cost of DR reserve	Multi-region may be cancelled	Lean bootstrap + configurable buffers + distributed DR + max-not-sum sizing
Two concurrent region failures	Overcommitted DR (DR-017) cannot cover both	Accept under DR-001; quantify via POC-011; conservative sum override (C-11) if a customer demands it
API throttling during regional event	Slow recovery	Bootstrap headroom + staged acquisition + backoff
Business volatility	Requirements churn	Configuration-driven design + living document
Two-region concentration	Cannot guarantee failover	Push for 3–4 regions per geography
Preview-feature dependency	Design lock delayed	Track DEP-001; design to work with and without sharing

25. Final Requirement Summary

The approved direction is a **dynamic, policy-driven Capacity & Quota Management Engine**, not a static pool of large DR reservations. The engine shall:

- protect production with **configurable** reservation and quota buffers;
- base reservations on **allocated VMs + buffer**;
- manage quota and reservations **separately** but correlate them before deployment;
- **pool and allocate** quota under central governance (quota-as-governor);
- maintain a **minimal, approved DR bootstrap** rather than a full production copy;
- use **CVAL and distributed regional capacity** to support single-region failover, with **reciprocal multi-source hosting** and **max-not-sum sizing**;
- provide **fresh state** to region selection and AEP;

- persist an authoritative **customer placement seed** and a **source→destination DR index**;
- support **reservation sharing** only after POC + enterprise approval;
- prevent **stale, double-committed, or unaudited** decisions;
- expose **cost, readiness, risk, and compliance** evidence; and
- retain **configuration flexibility** because regional, legal, product, and business policies change.

The remaining major architectural risks are **Capacity Reservation Sharing behaviour (POC-001)**, **DR subscription topology (POC-006)**, **bootstrap sizing (POC-007)**, **max-not-sum overcommit safety (POC-011)**, and **Middle East DR policy (DEC-001)**.

Appendix A — Core Formulas

A.1 Reservation target

Target Reserved Capacity = Allocated VM Count + Buffer Target

A.2 Reservation headroom

Reservation Headroom = Reserved Quantity - Allocated VM Count

A.3 Reservation deficit

Reservation Deficit = max(0, Target Reserved Capacity - Reserved Quantity)

A.4 Available subscription quota

Available Quota = Assigned Regional VM-Family Quota - Current Regional VM-Family Usage

A.5 Deployment quota deficit

Quota Deficit = max(0, Requested Deployment Units - Available Quota)

A.6 DR destination requirement (CORRECTED — max, not sum)

Destination DR Requirement(d)
 = MAX over each non-concurrent source region s protected by d (
 Workload Portion of s assigned to destination d
)

Rationale: under the single-region-failure assumption (DR-001), destination d never has to host more than one source region's failover at a time, so it need only be sized for its **largest** protected source — not the arithmetic sum of all of them. A conservative **SUM** override is available (C-11) only where a customer/contract explicitly requires protection against concurrent failures. See Appendix D for the full derivation and worked example.

A.7 DR capacity gap

DR Capacity Gap(d) = max(0, Destination DR Requirement(d) - Usable Destination Capacity(d))

Usable destination capacity may include approved bootstrap, available reservations, releasable CVAL capacity, and capacity acquired through approved sharing/expansion.

A.8 Shared-DR overcommit ratio (informational)

Overcommit Ratio(d) = SUM(source portions on d) / MAX(source portions on d)

A ratio > 1 quantifies the capacity (and cost) saved by max-not-sum sizing at destination d; it also equals the exposure if the single-failure assumption is violated.

Appendix B — Worked Cost Examples (illustrative)

These are the review-time back-of-envelope figures that drove the shift to a lean DR model. They are **illustrative planning numbers**, not billing quotes.

Scenario	Basis	Approx. cost
Reference block	500 × E32 16,000 cores 2 production subscriptions	—
30% empty DR reserve (reference block)	150 VMs held idle	~\$54K/month for the block
10% DR reserve (reference block)	50 VMs	lower, but still material
5% DR reserve (reference block)	~\$78K/year for 16,000 cores / 500×E32	“used = effectively free”
Heritage skeleton	1 VM of a specific SKU kept warm	~\$500K/year
Platform-wide 30% DR reserve	Scaled across ~10× the reference subscriptions	~\$1.5M–\$5M/year

Takeaway: a fixed 30–40% empty DR reserve is prohibitive at platform scale and would risk cancellation of multi-region. Bootstrap capacity that is *actually used* to stand up control planes is effectively free, so DR should hold only the minimum needed to bootstrap and scale via staged acquisition (DR-006).

Appendix C — Requirement Status Labels

- **Baseline** — agreed direction, suitable for design.
- **Configurable** — required capability; numeric value/policy not fixed.
- **POC validation required** — behaviour must be verified before production dependency.
- **Business decision required** — design must support alternatives until policy is approved.
- **External dependency** — depends on Azure feature maturity, Microsoft clarification, legal direction, or another platform team.

Appendix D — DR Sizing Formula Correction (Max vs Sum)

D.1 The problem with the original formula

Version 2.0 stated the destination requirement as:

Destination DR Requirement = Sum of Source Workload Portions Assigned to Destination over

This **sum** implicitly assumes that *every* source region a destination protects could fail **at the same time**, so the destination must hold enough standby capacity for **all of them simultaneously**. That directly contradicts the programme’s own planning assumption (DR-001): **only one region in a geography fails at a time**.

Sizing for the sum therefore buys capacity that, by our own design assumption, will **never be used concurrently** — reintroducing exactly the idle-reserve cost the lean bootstrap model was created to eliminate (FIN-006).

D.2 The corrected formula

Destination DR Requirement(d) = MAX over non-concurrent sources s protected by d (portion(s

Under single-failure, destination d only ever absorbs **one** source at a time, so it needs standby capacity for the **largest** source it protects — never the total. The standby “slots” are **shared** (overcommitted) across the mutually-exclusive failure events. This is the DR analogue of overbooking a resource that can only be claimed by one tenant at a time.

D.3 Worked example (from the reviewed diagram)

Suppose destination **Region 2** holds DR standby for customers whose production is in **Region 1** and **Region 3**:

Source protected by R2	Failover portion landing in R2 (E32 cores)
Region 1 (Cust1 + Cust5)	120
Region 3 (Cust6)	80

Old (sum) sizing:

Requirement(R2) = 120 + 80 = 200 cores reserved as standby

New (max) sizing:

Requirement(R2) = max(120, 80) = 120 cores reserved as standby

Saving at R2: 200 → 120 = **80 cores (40%) removed** with no loss of protection, because R1 and R3 do not fail together (DR-001).

Overcommit ratio (A.8):

Overcommit Ratio(R2) = 200 / 120 1.67

i.e., R2's shared standby is $1.67\times$ oversubscribed — the measure of both the saving *and* the exposure if two regions ever failed at once.

D.4 Scaling the saving

Extrapolating the Appendix B economics: if the naïve sum model implied ~\$1.5M–\$5M/year of platform-wide idle DR reserve, then in a 3-region geography where each destination protects two roughly comparable sources, max-not-sum sizing removes on the order of **~40–50% of the standby reserve** — potentially **\$0.6M–\$2.5M/year** — while still satisfying the single-failure SLA. Exact figures depend on portion distribution and SKUs and must be confirmed against real consumption (Roy's “reactive” balancing, PLC-007/DAT-001).

D.5 Guardrails on the correction

1. **Assumption-bound.** The max model is valid **only** while DR-001 (single-region failure) holds. If the business ever mandates concurrent-failure protection for a specific customer/geography, use the **SUM** override (C-11) for that scope.
2. **Quantify residual risk.** POC-011 validates activation behaviour of over-committed standby and quantifies the exposure (= overcommit ratio) so leadership signs off knowingly.
3. **Observe coverage.** OBS-001/002 must alert when a destination's usable capacity drops below its **max protected source**, not below the sum.
4. **Zone/region integrity preserved.** Max-not-sum changes only *how much* standby to hold per destination; it does **not** relax the region/zone isolation rules (CAP-011/CAP-012) or the separation constraints (ENV-003).
5. **Reversibility.** Because the basis is configurable (C-11), the estate can move between max and sum per scope without code changes (FIN-002).